Recursive least squares filter

From Wikipedia, the free encyclopedia

The **Recursive least squares (RLS)** is an adaptive filter which recursively finds the coefficients that minimize a weighted linear least squares cost function relating to the input signals. This is in contrast to other algorithms such as the least mean squares (LMS) that aim to reduce the mean square error. In the derivation of the RLS, the input signals are considered deterministic, while for the LMS and similar algorithm they are considered stochastic. Compared to most of its competitors, the RLS exhibits extremely fast convergence. However, this benefit comes at the cost of high computational complexity.

Contents

- 1 Motivation
- 2 Discussion
  - 2.1 Choosing λ
- 3 Recursive algorithm
- 4 RLS algorithm summary
- 5 Lattice recursive least squares filter (LRLS)
  - 5.1 Parameter Summary
  - 5.2 LRLS Algorithm Summary
- 6 Normalized lattice recursive least squares filter (NLRLS)
  - 6.1 NLRLS algorithm summary
- 7 See also
- 8 References
- 9 Notes

Motivation

RLS was discovered by Gauss but laid unused or ignored until 1950 when Plackett rediscovered the original work of Gauss from 1821. In general, the RLS can be used to solve any problem that can be solved by adaptive filters. For example, suppose that a signal d(n) is transmitted over an echoey, noisy channel that causes it to be received as

$$x(n) = \sum_{k=0}^{q} b_n(k) d(n-k) + v(n)$$

where $v(n)$ represents additive noise. We will attempt to recover the desired signal $d(n)$ by use of a $p+1$ -tap FIR filter, $\mathbf{w}$:

$$\hat{d}(n) = \sum_{k=0}^{p} w_n(k) x(n-k) = \mathbf{w}_n^T \mathbf{x}_n$$

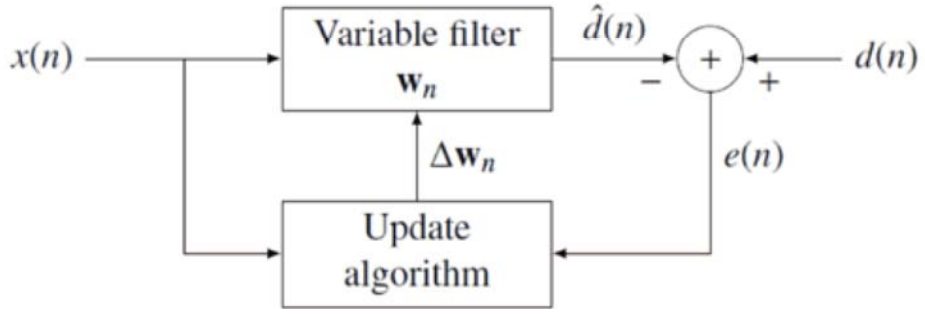
where $\mathbf{x}_n = [x(n) \quad x(n-1) \quad \dots \quad x(n-p)]^T$ is the vector containing the $p+1$ most recent samples of $x(n)$. Our goal is to estimate the parameters of the filter $\mathbf{w}$, and at each time n we refer to the new least squares estimate by $\mathbf{w}_n$. As time evolves, we would like to avoid completely redoing the least squares algorithm to find the new estimate for $\mathbf{w}_{n+1}$, in terms of $\mathbf{w}_n$.

The benefit of the RLS algorithm is that there is no need to invert matrices, thereby saving computational

power. Another advantage is that it provides intuition behind such results as the Kalman filter.

Discussion

The idea behind RLS filters is to minimize a cost function C by appropriately selecting the filter coefficients $\mathbf{w}_n$, updating the filter as new data arrives. The error signal $e(n)$ and desired signal $d(n)$ are defined in the negative feedback diagram below:



The error implicitly depends on the filter coefficients through the estimate $\hat{d}\,(n)$:

$$e(n) = d(n) - \hat{d}\,(n)$$

The weighted least squares error function C—the cost function we desire to minimize—being a function of e(n) is therefore also dependent on the filter coefficients:

$$C(\mathbf{w}_n) = \sum_{i=0}^{n} \lambda^{n-i} e^2(i)$$

where $0 < \lambda \leq 1$ is the "forgetting factor" which gives exponentially less weight to older error samples.

The cost function is minimized by taking the partial derivatives for all entries k of the coefficient vector $\mathbf{w}_n$ and setting the results to zero

$$\frac{\partial C(\mathbf{w}_n)}{\partial w_n(k)} = \sum_{i=0}^{n} 2\lambda^{n-i} e(i) \frac{\partial e(i)}{\partial w_n(k)} = -\sum_{i=0}^{n} 2\lambda^{n-i} e(i)\, x(i-k) = 0 \qquad k = 0, 1, \cdots, p$$

Next, replace $e(n)$ with the definition of the error signal

$$\sum_{i=0}^{n} \lambda^{n-i} \left[d(i) - \sum_{l=0}^{p} w_n(l) x(i-l) \right] x(i-k) = 0 \qquad k = 0, 1, \cdots, p$$

Rearranging the equation yields

$$\sum_{l=0}^{p} w_n(l) \left[\sum_{i=0}^{n} \lambda^{n-i}\, x(i-l) x(i-k) \right] = \sum_{i=0}^{n} \lambda^{n-i} d(i) x(i-k) \qquad k = 0, 1, \cdots, p$$

This form can be expressed in terms of matrices

$$\mathbf{R}_x(n)\, \mathbf{w}_n = \mathbf{r}_{dx}(n)$$

where $\mathbf{R}_x(n)$ is the weighted sample covariance matrix for $x(n)$, and $\mathbf{r}_{dx}(n)$ is the equivalent estimate for the cross-covariance between $d(n)$ and $x(n)$. Based on this expression we find the coefficients which minimize the cost function as

$$\mathbf{w}_n = \mathbf{R}_x^{-1}(n)\,\mathbf{r}_{dx}(n)$$

This is the main result of the discussion.

Choosing λ

The smaller λ is, the smaller is the contribution of previous samples to the covariance matrix. This makes the filter *more* sensitive to recent samples, which means more fluctuations in the filter co-efficients. The $\lambda = 1$ case is referred to as the *growing window RLS algorithm*. In practice, λ is usually chosen between 0.98 and 1.[1] By using type-II maximum likelihood estimation the optimal λ can be estimated from a set of data.[2]

Recursive algorithm

The discussion resulted in a single equation to determine a coefficient vector which minimizes the cost function. In this section we want to derive a recursive solution of the form

$$\mathbf{w}_n = \mathbf{w}_{n-1} + \Delta\mathbf{w}_{n-1}$$

where $\Delta\mathbf{w}_{n-1}$ is a correction factor at time $n-1$. We start the derivation of the recursive algorithm by expressing the cross covariance $\mathbf{r}_{dx}(n)$ in terms of $\mathbf{r}_{dx}(n-1)$

$$\begin{aligned}\mathbf{r}_{dx}(n) &= \sum_{i=0}^{n} \lambda^{n-i} d(i)\mathbf{x}(i) \\ &= \sum_{i=0}^{n-1} \lambda^{n-i} d(i)\mathbf{x}(i) + \lambda^0 d(n)\mathbf{x}(n) \\ &= \lambda\mathbf{r}_{dx}(n-1) + d(n)\mathbf{x}(n)\end{aligned}$$

where $\mathbf{x}(i)$ is the $p+1$ dimensional data vector

$$\mathbf{x}(i) = [x(i), x(i-1), \ldots, x(i-p)]^T$$

Similarly we express $\mathbf{R}_x(n)$ in terms of $\mathbf{R}_x(n-1)$ by

$$\begin{aligned}\mathbf{R}_x(n) &= \sum_{i=0}^{n} \lambda^{n-i} \mathbf{x}(i)\mathbf{x}^T(i) \\ &= \lambda\mathbf{R}_x(n-1) + \mathbf{x}(n)\mathbf{x}^T(n)\end{aligned}$$

In order to generate the coefficient vector we are interested in the inverse of the deterministic auto-covariance matrix. For that task the Woodbury matrix identity comes in handy. With

$A = \lambda\mathbf{R}_x(n-1)$ is $(p+1)$-by-$(p+1)$
$U = \mathbf{x}(n)$ is $(p+1)$-by-1
$V = \mathbf{x}^T(n)$ is 1-by-$(p+1)$
$C = \mathbf{I}_1$ is the 1-by-1 identity matrix

The Woodbury matrix identity follows

$$\begin{aligned}\mathbf{R}_x^{-1}(n) &= \left[\lambda\mathbf{R}_x(n-1) + \mathbf{x}(n)\mathbf{x}^T(n)\right]^{-1} \\ &= \lambda^{-1}\mathbf{R}_x^{-1}(n-1) \\ &\quad -\lambda^{-1}\mathbf{R}_x^{-1}(n-1)\mathbf{x}(n) \\ &\quad \left\{1 + \mathbf{x}^T(n)\lambda^{-1}\mathbf{R}_x^{-1}(n-1)\mathbf{x}(n)\right\}^{-1}\mathbf{x}^T(n)\lambda^{-1}\mathbf{R}_x^{-1}(n-1)\end{aligned}$$

To come in line with the standard literature, we define

$$\begin{aligned}\mathbf{P}(n) &= \mathbf{R}_x^{-1}(n) \\ &= \lambda^{-1}\mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\lambda^{-1}\mathbf{P}(n-1)\end{aligned}$$

where the *gain vector* $g(n)$ is

$$\begin{aligned}\mathbf{g}(n) &= \lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n)\left\{1 + \mathbf{x}^T(n)\lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n)\right\}^{-1} \\ &= \mathbf{P}(n-1)\mathbf{x}(n)\left\{\lambda + \mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{x}(n)\right\}^{-1}\end{aligned}$$

Before we move on, it is necessary to bring $\mathbf{g}(n)$ into another form

$$\begin{aligned}\mathbf{g}(n)\left\{1 + \mathbf{x}^T(n)\lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n)\right\} &= \lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n) \\ \mathbf{g}(n) + \mathbf{g}(n)\mathbf{x}^T(n)\lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n) &= \lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n)\end{aligned}$$

Subtracting the second term on the left side yields

$$\begin{aligned}\mathbf{g}(n) &= \lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n) - \mathbf{g}(n)\mathbf{x}^T(n)\lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n) \\ &= \lambda^{-1}\left[\mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\mathbf{P}(n-1)\right]\mathbf{x}(n)\end{aligned}$$

With the recursive definition of $\mathbf{P}(n)$ the desired form follows

$$\mathbf{g}(n) = \mathbf{P}(n)\mathbf{x}(n)$$

Now we are ready to complete the recursion. As discussed

$$\begin{aligned}\mathbf{w}_n &= \mathbf{P}(n)\,\mathbf{r}_{dx}(n) \\ &= \lambda\mathbf{P}(n)\,\mathbf{r}_{dx}(n-1) + d(n)\mathbf{P}(n)\,\mathbf{x}(n)\end{aligned}$$

The second step follows from the recursive definition of $\mathbf{r}_{dx}(n)$. Next we incorporate the recursive definition of $\mathbf{P}(n)$ together with the alternate form of $\mathbf{g}(n)$ and get

$$\begin{aligned}\mathbf{w}_n &= \lambda\left[\lambda^{-1}\mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\lambda^{-1}\mathbf{P}(n-1)\right]\mathbf{r}_{dx}(n-1) + d(n)\mathbf{g}(n) \\ &= \mathbf{P}(n-1)\mathbf{r}_{dx}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{r}_{dx}(n-1) + d(n)\mathbf{g}(n) \\ &= \mathbf{P}(n-1)\mathbf{r}_{dx}(n-1) + \mathbf{g}(n)\left[d(n) - \mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{r}_{dx}(n-1)\right]\end{aligned}$$

With $\mathbf{w}_{n-1} = \mathbf{P}(n-1)\mathbf{r}_{dx}(n-1)$ we arrive at the update equation

$$\begin{aligned}\mathbf{w}_n &= \mathbf{w}_{n-1} + \mathbf{g}(n)\left[d(n) - \mathbf{x}^T(n)\mathbf{w}_{n-1}\right] \\ &= \mathbf{w}_{n-1} + \mathbf{g}(n)\alpha(n)\end{aligned}$$

where $\alpha(n) = d(n) - \mathbf{x}^T(n)\mathbf{w}_{n-1}$ is the *a priori* error. Compare this with the *a posteriori* error; the error calculated *after* the filter is updated:

$$e(n) = d(n) - \mathbf{x}^T(n)\mathbf{w}_n$$

That means we found the correction factor

$$\Delta\mathbf{w}_{n-1} = \mathbf{g}(n)\alpha(n)$$

This intuitively satisfying result indicates that the correction factor is directly proportional to both the error and the gain vector, which controls how much sensitivity is desired, through the weighting factor, λ.

RLS algorithm summary

The RLS algorithm for a *p*-th order RLS filter can be summarized as

Parameters:	$p =$ filter order
	$\lambda =$ forgetting factor
	$\delta =$ value to initialize $\mathbf{P}(0)$
Initialization:	$\mathbf{w}(n) = 0,$
	$x(k) = 0, k = -p, \ldots, -1,$
	$d(k) = 0, k = -p, \ldots, -1$
	$\mathbf{P}(0) = \delta^{-1} I$ where I is the identity matrix of rank $p+1$
Computation:	For $n = 1, 2, \ldots$

$$\mathbf{x}(n) = \begin{bmatrix} x(n) \\ x(n-1) \\ \vdots \\ x(n-p) \end{bmatrix}$$

$$\alpha(n) = d(n) - \mathbf{x}^T(n)\mathbf{w}(n-1)$$
$$\mathbf{g}(n) = \mathbf{P}(n-1)\mathbf{x}^*(n)\left\{\lambda + \mathbf{x}^T(n)\mathbf{P}(n-1)\mathbf{x}^*(n)\right\}^{-1}$$
$$\mathbf{P}(n) = \lambda^{-1}\mathbf{P}(n-1) - \mathbf{g}(n)\mathbf{x}^T(n)\lambda^{-1}\mathbf{P}(n-1)$$
$$\mathbf{w}(n) = \mathbf{w}(n-1) + \alpha(n)\mathbf{g}(n).$$

Note that the recursion for P follows an Algebraic Riccati equation and thus draws parallels to the Kalman filter.[3]

Lattice recursive least squares filter (LRLS)

The **Lattice Recursive Least Squares** adaptive filter is related to the standard RLS except that it requires fewer arithmetic operations (order N). It offers additional advantages over conventional LMS algorithms such as faster convergence rates, modular structure, and insensitivity to variations in eigenvalue spread of the input correlation matrix. The LRLS algorithm described is based on *a posteriori* errors and includes the normalized form. The derivation is similar to the standard RLS algorithm and is based on the definition of

$d(k)$. In the forward prediction case, we have $d(k) = x(k)$ with the input signal $x(k-1)$ as the most up to date sample. The backward prediction case is $d(k) = x(k-i-1)$, where i is the index of the sample in the past we want to predict, and the input signal $x(k)$ is the most recent sample.[4]

Parameter Summary

$\kappa_f(k,i)$ is the forward reflection coefficient

$\kappa_b(k,i)$ is the backward reflection coefficient

$e_f(k,i)$ represents the instantaneous *a posteriori* forward prediction error

$e_b(k,i)$ represents the instantaneous *a posteriori* backward prediction error

$\xi^d_{b_{min}}(k,i)$ is the minimum least-squares backward prediction error

$\xi^d_{f_{min}}(k,i)$ is the minimum least-squares forward prediction error

$\gamma(k,i)$ is a conversion factor between *a priori* and *a posteriori* errors

$v_i(k)$ are the feedforward multiplier coefficients.

ϵ is a small positive constant that can be 0.01

LRLS Algorithm Summary

The algorithm for a LRLS filter can be summarized as

Initialization:

For i = 0,1,...,N

$\delta(-1,i) = \delta_D(-1,i) = 0$ (if x(k) = 0 for k < 0)

$\xi^d_{b_{min}}(-1,i) = \xi^d_{f_{min}}(-1,i) = \epsilon$

$\gamma(-1,i) = 1$

$e_b(-1,i) = 0$

End

Computation:

For k ≥ 0

$\gamma(k,0) = 1$

$e_b(k,0) = e_f(k,0) = x(k)$

$\xi^d_{b_{min}}(k,0) = \xi^d_{f_{min}}(k,0) = x^2(k) + \lambda\xi^d_{f_{min}}(k-1,0)$

$e(k,0) = d(k)$

For i = 0,1,...,N

$$\delta(k,i) = \lambda\delta(k-1,i) + \frac{e_b(k-1,i)e_f(k,i)}{\gamma(k-1,i)}$$

$$\gamma(k,i+1) = \gamma(k,i) - \frac{e_b^2(k,i)}{\xi^d_{b_{min}}(k,i)}$$

$$\kappa_b(k,i) = \frac{\delta(k,i)}{\xi^d_{f_{min}}(k,i)}$$

$$\kappa_f(k,i) = \frac{\delta(k,i)}{\xi^d_{b_{min}}(k-1,i)}$$

$$e_b(k,i+1) = e_b(k-1,i) - \kappa_b(k,i)e_f(k,i)$$

$$e_f(k,i+1) = e_f(k,i) - \kappa_f(k,i)e_b(k-1,i)$$

$$\xi^d_{b_{min}}(k,i+1) = \xi^d_{b_{min}}(k-1,i) - \delta(k,i)\kappa_b(k,i)$$

$$\xi^d_{f_{min}}(k,i+1) = \xi^d_{f_{min}}(k,i) - \delta(k,i)\kappa_f(k,i)$$

Feedforward Filtering

$$\delta_D(k,i) = \lambda\delta_D(k-1,i) + \frac{e(k,i)e_b(k,i)}{\gamma(k,i)}$$

$$v_i(k) = \frac{\delta_D(k,i)}{\xi^d_{b_{min}}(k,i)}$$

$$e(k,i+1) = e(k,i) - v_i(k)e_b(k,i)$$

End

End

Normalized lattice recursive least squares filter (NLRLS)

The normalized form of the LRLS has fewer recursions and variables. It can be calculated by applying a normalization to the internal variables of the algorithm which will keep their magnitude bounded by one. This is generally not used in real-time applications because of the number of division and square-root operations which comes with a high computational load.

NLRLS algorithm summary

The algorithm for a NLRLS filter can be summarized as

Initialization:

For i = 0,1,...,N

$\bar{\delta}(-1,i) = 0$ (if x(k) = d(k) = 0 for k < 0)

$\bar{\delta}_D(-1,i) = 0$

$\bar{e}_b(-1,i) = 0$

End

$\sigma^2_x(-1) = \lambda\sigma^2_d(-1) = \epsilon$

Computation:

For k ≥ 0

$\sigma^2_x(k) = \lambda\sigma^2_x(k-1) + x^2(k)$ (Input signal energy)

$\sigma^2_d(k) = \lambda\sigma^2_d(k-1) + d^2(k)$ (Reference signal energy)

$$\bar{e}_b(k,0) = \bar{e}_f(k,0) = \frac{x(k)}{\sigma_x(k)}$$

$$\bar{e}(k,0) = \frac{d(k)}{\sigma_d(k)}$$

For i = 0,1,...,N

$$\overline{\delta}(k,i) = \delta(k-1,i)\sqrt{(1-\overline{e}_b^2(k-1,i))(1-\overline{e}_f^2(k,i))} + \overline{e}_b(k-1,i)\overline{e}_f(k,i)$$

$$\overline{e}_b(k,i+1) = \frac{\overline{e}_b(k-1,i) - \overline{\delta}(k,i)\overline{e}_f(k,i)}{\sqrt{(1-\overline{\delta}^2(k,i))(1-\overline{e}_f^2(k,i))}}$$

$$\overline{e}_f(k,i+1) = \frac{\overline{e}_f(k,i) - \overline{\delta}(k,i)\overline{e}_b(k-1,i)}{\sqrt{(1-\overline{\delta}^2(k,i))(1-\overline{e}_b^2(k-1,i))}}$$

Feedforward Filter

$$\overline{\delta}_D(k,i) = \overline{\delta}_D(k-1,i)\sqrt{(1-\overline{e}_b^2(k,i))(1-\overline{e}^2(k,i))} + \overline{e}(k,i)\overline{e}_b(k,i)$$

$$\overline{e}(k,i+1) = \frac{1}{\sqrt{(1-\overline{e}_b^2(k,i))(1-\overline{\delta}_D^2(k,i))}}[\overline{e}(k,i) - \overline{\delta}_D(k,i)\overline{e}_b(k,i)]$$

End

End

See also

- Adaptive filter
- Kernel adaptive filter
- Least mean squares filter
- Zero forcing equalizer

References

- Hayes, Monson H. (1996). "9.4: Recursive Least Squares". *Statistical Digital Signal Processing and Modeling*. Wiley. p. 541. ISBN 0-471-59431-8.
- Simon Haykin, *Adaptive Filter Theory*, Prentice Hall, 2002, ISBN 0-13-048434-2
- M.H.A Davis, R.B. Vinter, *Stochastic Modelling and Control*, Springer, 1985, ISBN 0-412-16200-8
- Weifeng Liu, Jose Principe and Simon Haykin, *Kernel Adaptive Filtering: A Comprehensive Introduction*, John Wiley, 2010, ISBN 0-470-44753-2
- R.L.Plackett, *Some Theorems in Least Squares*, Biometrika, 1950, 37, 149-157, ISSN 0006-3444 (https://www.worldcat.org/search?fq=x0:jrnl&q=n2:0006-3444)
- C.F.Gauss, *Theoria combinationis observationum erroribus minimis obnoxiae*, 1821, Werke, 4. Gottinge

Notes

1. Emannual C. Ifeacor, Barrie W. Jervis. Digital signal processing: a practical approach, second edition. Indianapolis: Pearson Education Limited, 2002, p. 718
2. Steven Van Vaerenbergh, Ignacio Santamaría, Miguel Lázaro-Gredilla "Estimation of the forgetting factor in kernel recursive least squares" (http://gtas.unican.es/files/pub/stkrls_mlsp2012.pdf), 2012 IEEE International Workshop on Machine Learning for Signal Processing, 2012, accessed June 23, 2016.
3. Welch, Greg and Bishop, Gary "An Introduction to the Kalman Filter" (http://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf), Department of Computer Science, University of North Carolina at Chapel Hill, September 17, 1997, accessed July 19, 2011.
4. Albu, Kadlec, Softley, Matousek, Hermanek, Coleman, Fagan "Implementation of (Normalised) RLS Lattice on Virtex" (http://wwwdsp.ucd.ie/dspfiles/main_files/pdf_files/hsla_fpl2001.pdf), Digital Signal Processing, 2001, accessed December 24, 2011.

Retrieved from "https://en.wikipedia.org/w/index.php?
title=Recursive_least_squares_filter&oldid=726637371"

Categories: Digital signal processing | Filter theory | Time series analysis

---

- This page was last modified on 23 June 2016, at 11:40.
- Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.